

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("uciml/breast-cancer-wisconsin-data")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'breast-cancer-wisconsin-data' dataset.
Path to dataset files: /kaggle/input/breast-cancer-wisconsin-data

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
import numpy as np

# Initialize and train Random Forest Classifier
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
y_prob_rf = rf_model.predict_proba(X_test)[:, 1]

# Initialize and train Gradient Boosting Classifier
gbc_model = GradientBoostingClassifier(random_state=42)
gbc_model.fit(X_train, y_train)
y_pred_gbc = gbc_model.predict(X_test)
y_prob_gbc = gbc_model.predict_proba(X_test)[:, 1]

# Initialize and train XGBoost Classifier
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
y_pred_xgb = xgb_model.predict(X_test)
y_prob_xgb = xgb_model.predict_proba(X_test)[:, 1]

print("Models trained successfully.")
```

Models trained successfully.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [05:55:02] WARNING:
Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Assuming the dataset path contains 'data.csv'
# If this path is incorrect, please let me know or modify the path variable.
df = pd.read_csv(f"{path}/data.csv")

# Display the first 5 rows and information about the dataframe
display(df.head())
display(df.info())
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	com
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	

5 rows × 33 columns

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 569 entries, 0 to 568

Data columns (total 33 columns):

#	Column	Non-Null Count	Dtype
0	id	569 non-null	int64
1	diagnosis	569 non-null	object
2	radius_mean	569 non-null	float64
3	texture_mean	569 non-null	float64
4	perimeter_mean	569 non-null	float64
5	area_mean	569 non-null	float64
6	smoothness_mean	569 non-null	float64
7	compactness_mean	569 non-null	float64
8	concavity_mean	569 non-null	float64
9	concave points_mean	569 non-null	float64
10	symmetry_mean	569 non-null	float64
11	fractal_dimension_mean	569 non-null	float64
12	radius_se	569 non-null	float64
13	texture_se	569 non-null	float64
14	perimeter_se	569 non-null	float64
15	area_se	569 non-null	float64
16	smoothness_se	569 non-null	float64
17	compactness_se	569 non-null	float64
18	concavity_se	569 non-null	float64
19	concave points_se	569 non-null	float64
20	symmetry_se	569 non-null	float64
21	fractal_dimension_se	569 non-null	float64
22	radius_worst	569 non-null	float64
23	texture_worst	569 non-null	float64
24	perimeter_worst	569 non-null	float64
25	area_worst	569 non-null	float64
26	smoothness_worst	569 non-null	float64
27	compactness_worst	569 non-null	float64
28	concavity_worst	569 non-null	float64
29	concave points_worst	569 non-null	float64
30	symmetry_worst	569 non-null	float64
31	fractal_dimension_worst	569 non-null	float64
32	Unnamed: 32	0 non-null	float64

dtypes: float64(31), int64(1), object(1)

memory usage: 146.8+ KB

None

```
# Drop unnecessary columns
```

```
df = df.drop(['id', 'Unnamed: 32'], axis=1)
```

```
# Convert diagnosis to numerical (M=1, B=0)
```

```
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})
```

```
# Separate features (X) and target (y)
```

```
X = df.drop('diagnosis', axis=1)
```

```

y = df['diagnosis']

# Scale the features using StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert scaled features back to a DataFrame for better readability
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

display(X_scaled_df.head())
print(f"Shape of scaled features: {X_scaled.shape}")

```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concav
0	1.097064	-2.073335	1.269934	0.984375	1.568466	3.283515	
1	1.829821	-0.353632	1.685955	1.908708	-0.826962	-0.487072	
2	1.579888	0.456187	1.566503	1.558884	0.942210	1.052926	
3	-0.768909	0.253732	-0.592687	-0.764464	3.283553	3.402909	
4	1.750297	-1.151816	1.776573	1.826229	0.280372	0.539340	

5 rows × 30 columns

Shape of scaled features: (569, 30)

```

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

```

Shape of X_train: (398, 30)

Shape of X_test: (171, 30)

Shape of y_train: (398,)

Shape of y_test: (171,)

```

# Evaluate Random Forest
accuracy_rf = accuracy_score(y_test, y_pred_rf)
roc_auc_rf = roc_auc_score(y_test, y_prob_rf)
print(f"Random Forest - Accuracy: {accuracy_rf:.4f}, ROC-AUC: {roc_auc_rf:.4f}")

# Evaluate Gradient Boosting
accuracy_gbc = accuracy_score(y_test, y_pred_gbc)
roc_auc_gbc = roc_auc_score(y_test, y_prob_gbc)
print(f"Gradient Boosting - Accuracy: {accuracy_gbc:.4f}, ROC-AUC: {roc_auc_gbc:.4f}")

# Evaluate XGBoost
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
roc_auc_xgb = roc_auc_score(y_test, y_prob_xgb)
print(f"XGBoost - Accuracy: {accuracy_xgb:.4f}, ROC-AUC: {roc_auc_xgb:.4f}")

# Plot ROC Curves
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)
fpr_gbc, tpr_gbc, _ = roc_curve(y_test, y_prob_gbc)
fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_prob_xgb)

```

```
plt.figure(figsize=(10, 8))
plt.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {roc_auc_rf:.2f})')
plt.plot(fpr_gbc, tpr_gbc, label=f'Gradient Boosting (AUC = {roc_auc_gbc:.2f})')
plt.plot(fpr_xgb, tpr_xgb, label=f'XGBoost (AUC = {roc_auc_xgb:.2f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison')
plt.legend()
plt.grid(True)
plt.show()
```

Random Forest - Accuracy: 0.9708, ROC-AUC: 0.9969
Gradient Boosting - Accuracy: 0.9591, ROC-AUC: 0.9951
XGBoost - Accuracy: 0.9649, ROC-AUC: 0.9950



